Intelligent Warehouse Slotting Optimization

Goal:
Build a system that decides the best place to store products in a warehouse so workers can pick orders faster, travel less, and handle items safely.

Main Rules:

- Fast-moving products near dispatch area
- Heavy products in safe storage positions
- Frequently bought together products near each other

1. Description

Warehouses store thousands of products on racks and shelves. If products are not placed smartly, workers spend more time walking around to pick items for orders.

In this problem, students have to build a system that decides **where each product should be placed inside the warehouse** so that picking becomes faster, safer, and more efficient.

The goal is to create an **intelligent slotting plan** that reduces travel time for warehouse staff.

2. Problem Background

Imagine you are working in a **big warehouse**.

This warehouse has **thousands of products** — like phones, clothes, snacks, and more.

Now, a customer places an order.

You get a list of items and you have to go inside the warehouse and pick them.

But here's the problem...

👉 Products are placed **randomly**
👉 Some popular items are kept **very far away**

    - 👉 Heavy items are placed in **unsafe locations**
    - 👉 Items that are usually bought together are in **completely different areas**
- So what happens?
    - 🚶 You walk a lot…
    - 😓 You get tired…
    - ⏱️ Orders take more time…
    - 💸 Company loses efficiency…

Now imagine this instead 👇

What if…

- The **most frequently ordered products** are kept **near the dispatch area**
- **Heavy products** are placed safely and easy to pick
- Items that are often bought together are **placed next to each other**

Now what happens?

🚀 You pick items faster
🚶 Less walking
😊 Less effort
📦 Faster delivery

This is called **warehouse slotting optimization**.

3. Problem Statement

Build a system that recommends the **best storage location for each product in a warehouse**.

The system should help warehouse teams:

- reduce picker travel distance
- improve picking speed
- store products more safely
- place related items near each other

The solution should generate an **optimal slotting plan** and show how much travel distance can be reduced compared to a basic or random placement.

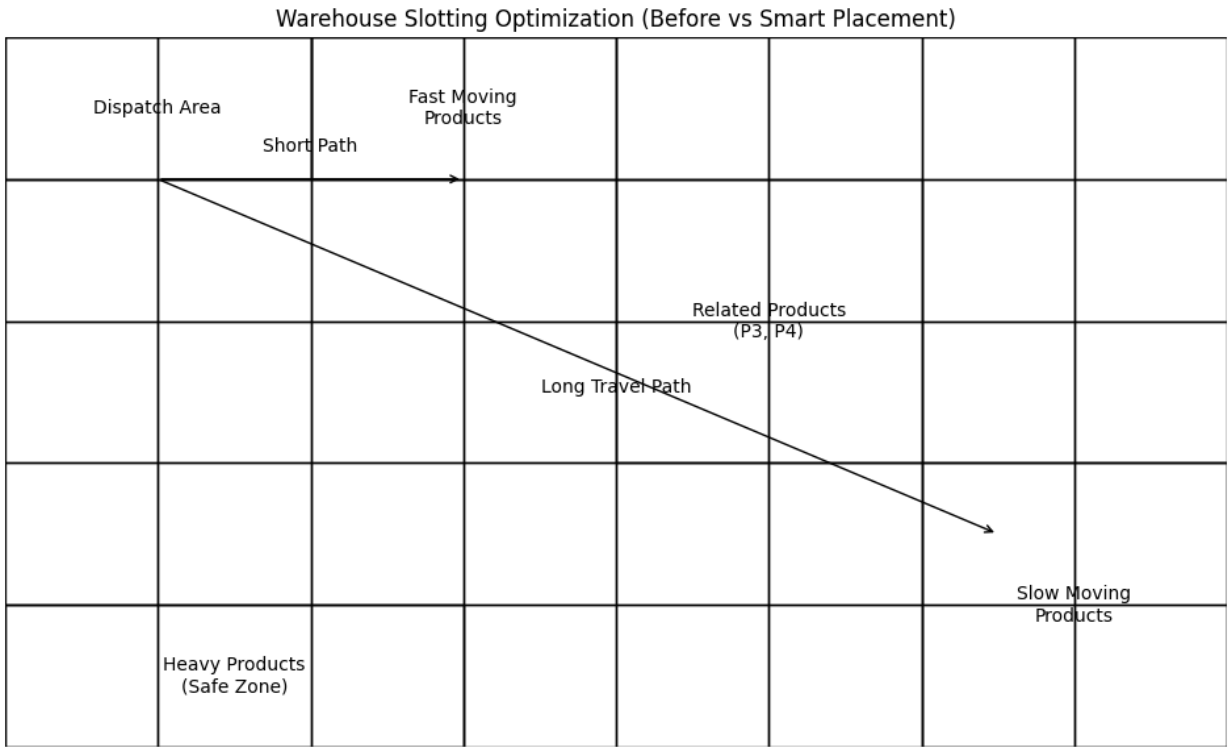

Diagram description:

- **Top-left = Dispatch Area** → starting point for picking
- **Fast-moving products** placed nearby → short path
- **Heavy products** placed at bottom → safe handling
- **Related products grouped together** → fewer trips
- **Slow-moving products placed far** → less priority

Key visual insight:

- **Long arrow** → shows bad layout (more walking 😓)
- **Short arrow** → shows optimized layout (less walking 🚀)

4. Key Requirements

The solution should consider these business rules:

Product placement rules

- **Fast-moving products** should be placed closer to the dispatch area
- **Heavy products** should be stored in safe locations
- **Frequently bought together products** should be placed near each other

System requirements

- Accept warehouse data such as:
    - product list
    - product demand/frequency
    - product weight
    - warehouse slot/rack locations
    - order history
- Recommend the best slot for each product
- Compare current layout vs optimized layout
- Show estimated travel distance savings
- Display results in a simple dashboard

Optional advanced ideas

- Re-slot products when demand changes
- Handle limited storage capacity
- Add zone-based or category-based placement
- Support multiple dispatch areas

5. What a Winning Solution Looks Like

A winning solution should be:

Simple: Easy to understand and easy to use.

Practical: Looks like something that can actually be used in a real warehouse.

Data-driven: Uses product movement, order history, and item weight to make smart decisions.

Optimized: Clearly reduces picker travel distance compared to random or current placement.

Explainable: Shows *why* a product was placed in a certain location.

Visual: Includes warehouse maps, slot assignment views, and comparison charts.

Strong winning features

A very good solution may include:

- smart slot recommendation algorithm
- travel distance simulation before and after optimization
- clustering of products often bought together
- safety rules for heavy items
- interactive dashboard
- ability to test different layouts

Main Rules:

- Fast-moving products near dispatch area
- Heavy products in safe storage positions
- Frequently bought together products near each other

Expected Output:

- Optimized slotting plan
- Comparison with current/random placement
- Travel distance reduction simulation
- Simple dashboard for warehouse insights

6. Dashboard Requirements

The dashboard should be simple and visual. It can have the following sections:

A. Warehouse Overview

- Total number of products
- Total number of slots
- Occupied vs free slots
- Number of fast-moving products
- Number of heavy products

B. Current vs Optimized Layout

- Current slotting plan
- Recommended slotting plan
- Product-to-slot mapping

C. Travel Distance Analysis

- Total picking distance in current layout
- Total picking distance in optimized layout
- Distance reduction in percentage
- Average picking time saved

D. Product Insights

- Fast-moving products
- Heavy products
- Frequently bought together products
- Products with highest picking frequency

E. Visual Warehouse Map

- Warehouse grid/rack view
- Color coding for product type or movement speed
- Highlight dispatch area

- Show where each product is placed

F. Simulation Section

- Run simulation for sample orders
- Compare picker routes before and after optimization
- Show step-by-step travel path

G. Alerts / Recommendations

- Unsafe placement warnings
- Overstocked or unused slots
- Suggestions for re-slotting

7. Database Schema

Below is a simple database design students can use.

1. Products

Stores product details.

Column Name	Type	Description
product_id	INT / VARCHAR	Unique product ID
product_name	VARCHAR	Product name
category	VARCHAR	Product category
weight	FLOAT	Product weight
demand_score	FLOAT	How fast the product moves
size	VARCHAR	Small/Medium/Large
is_fast_moving	BOOLEAN	Whether product is fast-moving

2. Warehouse_Slots

Stores all warehouse slot details.

Column Name	Type	Description
slot_id	INT / VARCHAR	Unique slot ID
zone	VARCHAR	Warehouse zone
rack_no	VARCHAR	Rack number
shelf_no	VARCHAR	Shelf number
x_coordinate	FLOAT	X position in warehouse
y_coordinate	FLOAT	Y position in warehouse
distance_to_dispatch	FLOAT	Distance from dispatch area
max_weight_capacity	FLOAT	Max weight allowed
slot_status	VARCHAR	Occupied / Empty

3. Orders

Stores customer order details.

Column Name	Type	Description
order_id	INT / VARCHAR	Unique order ID
order_date	DATE	Date of order
dispatch_time	DATETIME	Dispatch time

4. Order_Items

Stores products inside each order.

Column Name	Type	Description
order_item_id	INT / VARCHAR	Unique row ID
order_id	INT / VARCHAR	Linked order ID
product_id	INT / VARCHAR	Linked product ID
quantity	INT	Quantity ordered

5. Product_Affinity

Stores products that are frequently bought together.

Column Name	Type	Description
affinity_id	INT / VARCHAR	Unique row ID
product_id_1	INT / VARCHAR	First product
product_id_2	INT / VARCHAR	Second product
affinity_score	FLOAT	Strength of co-purchase relationship

6. Slot_Assignment

Stores current or recommended product placement.

Column Name	Type	Description
assignment_id	INT / VARCHAR	Unique row ID
product_id	INT / VARCHAR	Product ID
slot_id	INT / VARCHAR	Assigned slot ID
assignment_type	VARCHAR	Current / Optimized
assigned_on	DATETIME	Assignment date

7. Simulation_Results

Stores output of travel distance simulation.

Column Name	Type	Description
simulation_id	INT / VARCHAR	Unique simulation ID
layout_type	VARCHAR	Current / Optimized
total_distance	FLOAT	Total travel distance
avg_distance_per_order	FLOAT	Average picking distance
distance_reduction_percent	FLOAT	Improvement percentage
run_date	DATETIME	Simulation run date